

NAME: TANKISO MASOEBE

COMPUTER ARCHITECTURE WK7

1.Scenario Analysis: Cache Design for a Database Server

A database server can use a cache design to improve performance and reduce the number of requests sent to storage. The cache is a fast memory area, such as RAM, that temporarily stores frequently accessed data. When a user requests information, the database first checks the cache. If the required data is available, a cache hit occurs, and the data is returned quickly. If the data is not available, a cache miss occurs, so the database retrieves it from permanent storage and places a copy in the cache. A Least Recently Used (LRU) replacement policy can be used to remove older or less frequently accessed data when the cache becomes full. The system should also use cache expiration to ensure that outdated information is removed. This cache design improves response time, reduces disk input/output operations, lowers server workload, and allows the database to handle more users efficiently.

2.CONCEPT RESEARCH:

Cache Coherence

Cache coherence is a computer architecture technique that ensures multiple caches contain consistent and up-to-date copies of shared data. It is especially important in systems with multiple processors or CPU cores, where each processor may have its own cache. When one processor changes data, the cache coherence system ensures that other caches are notified or updated so they do not use outdated information. Common cache coherence protocols include MESI, which uses states such as Modified, Exclusive, Shared, and Invalid. Cache coherence improves system reliability and ensures that processors work with accurate data. It is essential for efficient and correct multiprocessing and parallel computing systems.

3.TOOL PRACTICE:

Cache Simulation in Visual Studio Code

I simulated a simple cache in Visual Studio Code using a small program that stores frequently accessed data in memory. The simulation used a key-value structure to represent cached items and checked the cache before retrieving data from the main storage. When data was found in the cache, a cache hit occurred, providing faster access. When data was not found, a cache miss occurred, and the program retrieved and stored the data in the cache. This practice helped me understand how caching improves system performance. I learned that caching reduces access time, minimizes workload, and improves application responsiveness.

4.DIAGRAM:

Cache Optimization Strategy

A cache optimization strategy is designed to improve system performance by storing frequently accessed data in fast memory. The strategy will use a Least Recently Used (LRU) replacement policy to remove data that has not been accessed recently when the cache becomes full. Frequently requested data will remain available, reducing the need to access slower storage. Cache expiration will also be applied to remove outdated information and maintain data accuracy. The system will monitor cache hits and misses to identify performance problems and adjust the cache size when necessary. This strategy reduces response time, lowers system workload, improves resource utilization, and provides faster access to frequently requested data.

